Initial Project Idea

ECE 382V, Fall 2026

Are LLM Written Unit Tests Flaky?

We want to find out whether unit tests written by LLMs are flaky, meaning they can pass and fail on the same code without anything changing. Tools like Copilot and research tools like TestPilot usually generate a test, run it once, and keep it if it passes, so a flaky test can slip in and only start failing later in CI. Most of the flaky test studies we have read look at tests written by people, so it is not clear how LLM written tests compare. Our idea is to take a set of small Python libraries and have a few LLM setups write tests for them, including an agent that can run and fix its own tests and models that write tests without ever running them. We would then rerun those suites and the human written tests that come with each library many times while changing the test order, the hash seed, the time zone, and the CPU load, and count a test as flaky if it both passes and fails. To evaluate this we would compare flaky rates between the LLM and human tests, label the root cause of each flaky test with the Luo et al. categories, and check whether simple patterns like real sleeps or unfrozen clock reads predict which tests flip. If the LLM tests turn out to be riskier, we want to try a fix such as a prompt that warns about these patterns or rerunning new tests a few times before keeping them. A small pilot on a couple of libraries already found an LLM written test that only fails after 10 PM. The main thing we are unsure about is that flaky tests are rare, so we may need a lot of reruns or a slower machine to see real differences.

Team

Shaunak Kapur (sk62288) and Manas Pathak (map7848)

Are LLM Written Unit Tests Flaky?

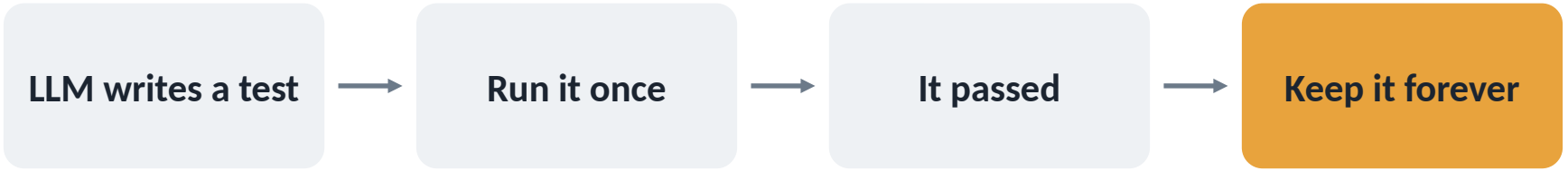
Initial project idea for ECE 382V

Shaunak Kapur and Manas Pathak



30 reruns of one test, one of them failed

Generation pipelines trust one passing run



- TestPilot and most Copilot style pipelines keep a generated test if it passes one run.
- Every big flaky test study so far looks at tests written by people.
- So we want to know if LLM tests are more flaky, why, and if one run is enough to catch it.



A flaky test can pass the one run that decides if it gets kept.

What we plan to run

Candidate Python libraries

cachetools	schedule	tinydb
humanize	toolz	tabulate
tenacity	filelock	boltons
more-itertools	arrow	pybreaker

Tan ones use time, randomness, threads, or files, which is where flaky tests usually come from.

4 test suites per library

Human
the library's own test suite

Agentic Sonnet
runs its tests and fixes failures

One shot Sonnet
writes tests, never runs them

One shot Haiku
same, with a smaller model

Each suite gets rerun many times with

- random order and hash seed
- 10 time zones
- CPU stress on every core

What a small pilot found

A test that only fails after 10 PM

```
t = now() + 2 hours
job.until(t.strftime("%H:%M:%S"))
```

After 10 PM that clock time lands earlier today, so the library rejects it as a time in the past. An LLM wrote it, and it passed the one run that would decide if it gets kept.

Random retry delay

Asserts two random retry waits add up to at least 10 ms. Sometimes they do not.

Tight deadline

Expects a retry loop to finish within 120 ms. Breaks when the CPU is busy.

Human tests too

Some human tests only fail in certain orders because an earlier test leaves something global behind.

Next steps and open questions

How we will evaluate

- Flaky rate per suite with confidence intervals
- Root cause of every flaky test
- Risk patterns and coverage per suite
- What a one run filter keeps and throws away

Timeline

- Oct 9 proposal
- Oct 23 more models and reruns
- Nov 6 slower CI machine runs
- Nov 20 mitigation prompt
- Dec 12 final report

What we are unsure about

- Flips are rare, so how many reruns are enough
- Do risk patterns predict real flips
- Is a laptop too fast to show timing bugs
- Ideas for other perturbations welcome